



MÓDULO 1

Por qué la IA escribe código que no puedes controlar

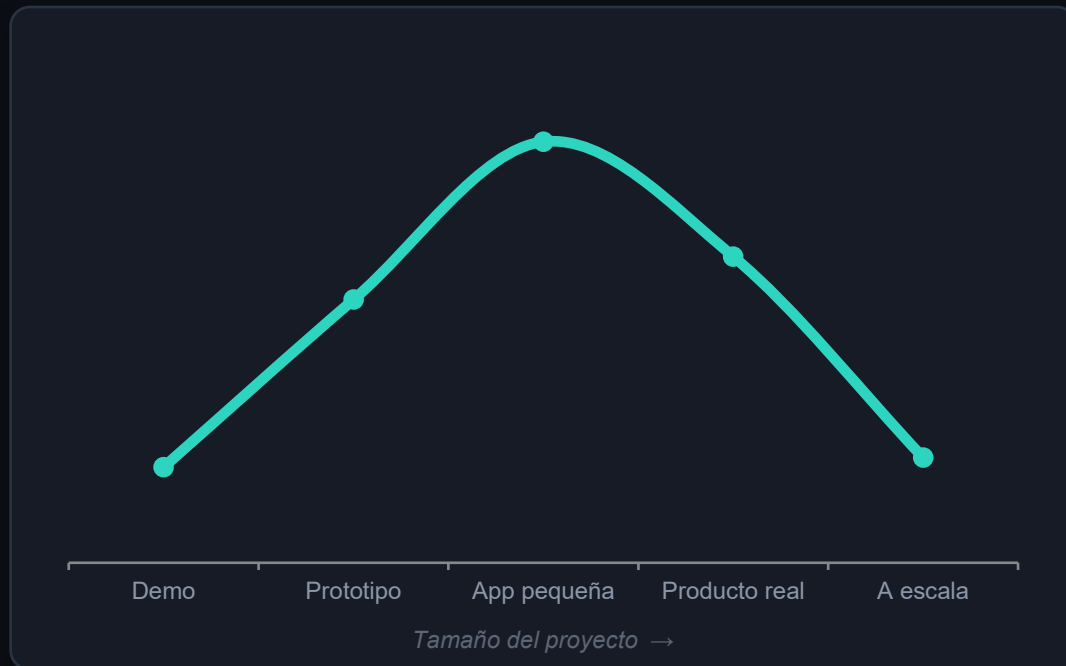
El problema que resuelve el desarrollo dirigido por especificaciones

CÓMO LLEGAMOS AQUÍ

De Copilot al vibe coding



El vibe coding funciona... hasta que no



Velocidad de avance

Para una demo o un prototipo de fin de semana, va de maravilla.

Cuando el proyecto crece (usuarios, datos y dinero reales) se derrumba.

Esa subida fácil es exactamente la trampa.

APRÉNDETE ESTOS TRES

Los 3 fallos del vibe coding



01

Deriva de intención

Tú pensabas una cosa; la IA entendió otra parecida. Como nunca lo escribiste, cada petición se aleja un poco más de tu idea.



02

Invención

Cuando le falta información, no pregunta: rellena los huecos con suposiciones que suenan bien. Y lo inventado siempre parece correcto.



03

Degradación

Cada petición nueva puede romper algo anterior: no hay nada que diga “esto es intocable”. Una torre de Jenga cada vez más alta.



EL GIRO DE 2025-2026

The spec is the prompt

El código deja de ser la fuente de verdad. La especificación (escrita en tu idioma, no en Código) define qué construyes, por qué, con qué reglas y cómo se comprueba. No le hablas a la IA con frases sueltas: le entregas un contrato.

Spec → Plan → Tareas → Código

Misma petición, resultados muy distintos

PROMPT VAGO

“Hazme una app para gestionar presupuestos de clientes.”

La IA tiene que adivinar:

¿Qué es un presupuesto?

¿Qué estados existen?

¿Qué se valida?

¿Qué no se rompe?

Rápido al inicio · Incontrolable después

MISMA IA

PROMPT CON SPEC

SPEC · PresupuestosPro

OBJETIVO Enviar presupuestos

DATOS Cliente, líneas, IVA, total

REGLAS Borrador → Enviado → Aceptado

CRITERIOS Recalcular total ·
bloquear aceptar sin cliente

Más explícito · Mucho menos caos

QUÉ ES REALMENTE UNA SPEC

No es un prompt largo: es un contrato

EL CONTRATO

SPEC

La fuente de verdad: el agente la lee, la sigue y comprueba cada paso contra ella.



01

Problema
y usuario

02

Datos
y pantallas

03

No objetivos

04

Reglas de
negocio

05

Criterios de
aceptación

06

Preguntas
abiertas

Idea difusa → SPEC → Plan → Tareas → Código comprobable

Loop engineering necesita raíles

Ya no prompteas turno a turno: diseñas el **bucle** que **promptea** al agente por ti.

↺ EL LOOP SE REPITE EN CADA VUELTA



LA SPEC = EL VERIFICADOR. Lo difícil no es iterar el loop: es saber cuándo terminó bien.

! Sin spec, el loop automatiza la deriva. Con ella, el agente **PARA**, pregunta, actualiza y repite.

De fallo a antídoto

EL FALLO



Deriva de intención



Invención



Degradación



EL ANTÍDOTO DE LA SPEC

La intención queda **ESCRITA**. Si el resultado se desvía, hay un documento contra el que comparar.

La spec hace explícito lo implícito: el agente **PREGUNTA** antes de escribir, en vez de suponer.

La spec marca qué es **intocable**; sus criterios se vuelven pruebas que vigilan el proyecto para siempre.

NO ES UNA OPINIÓN: ES LA INDUSTRIA

El panorama SDD en la actualidad

● GitHub Spec Kit

Gratis y abierto. El flujo de este curso.

● AWS Kiro

Editor donde todo nace de specs.

● OpenSpec

Abierto y agnóstico · 20+ agentes.

● Claude Code

Nuestro agente constructor.

● Cursor

Editor con reglas (.cursor/rules).

● BMAD-METHOD

Orquestación por roles.

Competidores feroces, el MISMO flujo. No aprendes una herramienta: aprendes el método (portable entre todas).